CHANGED 3 MONTHS AGO

🗓️ Buổi 8: Câu điều kiện, vòng lặp, hàm, promise, async await

> if, else, else if, switch
> Vòng lặp: for, while, do...while
> Hàm: định nghĩa, tham số, giá trị trả về
> Hàm ẩn danh, arrow function
> Promise, async await (Mở rộng ra Promise.all)

📘 PHẦN 1: CẤU TRÚC ĐIỀU KIỆN

🔎 1. Khái niệm lý thuyết
Trong lập trình, cấu trúc điều kiện giúp chương trình quyết định nên thực hiện đoạn mã nào dựa trên một điều kiện đúng hoặc sai.

Ví dụ đời thường:
Nếu trời mưa thì mang theo ô, nếu không thì không cần.

✅ 2. Câu lệnh if, else
🔹 Khái niệm:
Dùng để kiểm tra điều kiện. Nếu điều kiện đúng thì thực hiện một hành động, nếu sai thì thực hiện hành động khác.

🔹 Cú pháp:

```
if (điều_kiện) {
  // mã được thực hiện nếu điều kiện đúng
} else {
  // mã được thực hiện nếu điều kiện sai
}
```

🔹 Ví dụ:

```
let age = 16;

if (age >= 18) {
  console.log("Đủ tuổi bầu cử");
} else {
  console.log("Chưa đủ tuổi");
}
```

✅ 3. Câu lệnh else if
🔹 Khái niệm:
Dùng để kiểm tra nhiều điều kiện khác nhau, từ trên xuống dưới.

🔹 Cú pháp:

```
if (điều_kiện_1) {
  // thực hiện nếu điều kiện 1 đúng
} else if (điều_kiện_2) {
  // thực hiện nếu điều kiện 1 sai, điều kiện 2 đúng
} else {
  // thực hiện nếu tất cả điều kiện đều sai
}
```

🔹 Ví dụ:

```
let score = 75;
```

```
if (score >= 90) {
  console.log("Giỏi");
} else if (score >= 70) {
  console.log("Khá");
} else if (score >= 50) {
  console.log("Trung bình");
} else {
  console.log("Yếu");
}
```

✅ 4. Câu lệnh switch

🔹 Khái niệm:

switch giúp kiểm tra một giá trị cụ thể với nhiều trường hợp (case) khác nhau. Thường dùng để thay thế if...else if với các giá trị rời rạc.

🔹 Cú pháp:

```
switch (biến) {
  case giá_trị_1:
    // hành động khi khớp giá_trị_1
    break;
  case giá_trị_2:
    // hành động khi khớp giá_trị_2
    break;
  default:
    // hành động khi không khớp giá trị nào
}
```

🔹 Ví dụ:

```
let day = 2;

switch (day) {
  case 1:
    console.log("Thứ Hai");
    break;
  case 2:
    console.log("Thứ Ba");
    break;
  case 3:
    console.log("Thứ Tư");
    break;
  default:
    console.log("Không hợp lệ");
}
```

📝 Bài tập áp dụng

- Viết chương trình nhập một số, kiểm tra số đó là chẵn hay lẻ.
- Nhập điểm thi, phân loại học lực (>= 90: Giỏi, 70-89: Khá, 50-69: TB, <50: Yếu).

🔁 PHẦN 2: VÒNG LẶP

🔎 1. Khái niệm lý thuyết

Vòng lặp cho phép lặp lại một đoạn mã nhiều lần. Thay vì viết 10 dòng giống nhau, ta chỉ cần viết 1 vòng lặp.

✅ 2. Vòng lặp for

🔹 Khái niệm:

Dùng khi biết số lần lặp cụ thể.

🔹 Cú pháp:

```
for (khởi_tạo; điều_kiện; cập_nhật) {
  // thân vòng lặp
}
```

🔹 Ví dụ:

```
for (let i = 1; i <= 5; i++) {
```

```
  console.log("Lần lặp thứ: " + i);
}
```

✅ 3. Vòng lặp while

🔹 Khái niệm:

Dùng khi chưa biết trước số lần lặp, chỉ lặp khi điều kiện còn đúng.

🔹 Cú pháp:

```
while (điều_kiện) {
  // thân vòng lặp
}
```

🔹 Ví dụ:

```
let i = 1;
while (i <= 3) {
  console.log(i);
  i++;
}
```

✅ 4. Vòng lặp do...while

🔹 Khái niệm:

Giống while, nhưng luôn thực hiện ít nhất 1 lần trước khi kiểm tra điều kiện.

🔹 Cú pháp:

```
do {
  // thân vòng lặp
} while (điều_kiện);
```

🔹 Ví dụ:

```
let i = 5;
do {
  console.log("Giá trị i: " + i);
  i++;
} while (i <= 3);
```

📝 Bài tập áp dụng

- Dùng for để in bảng cửu chương của số 6.
- Dùng while để tính tổng các số từ 1 đến 100.
- Dùng do...while để yêu cầu người dùng nhập đúng mật khẩu "123456" mới cho thoát vòng lặp.

📘 PHẦN 2: HÀM TRONG JAVASCRIPT

🔎 1. Khái niệm lý thuyết

✅ Hàm là gì?

- Hàm (function) là một tập hợp các câu lệnh được gom nhóm lại và đặt tên.
- Mục đích: giúp tái sử dụng mã, chia nhỏ chương trình thành các phần dễ quản lý.
- Bạn có thể gọi lại hàm bất kỳ lúc nào bạn muốn.

.

🧩 2. Khai báo và sử dụng hàm

✅ Cú pháp:

```
function tênHàm() {
  // các câu lệnh
}

// Gọi hàm
```

```
tênHàm();
```

🔹 Ví dụ:

```
function sayHello() {
  console.log("Xin chào!");
}

sayHello(); // Gọi hàm
```

🧩 3. Tham số (Parameters)
✅ Khái niệm:
Tham số là biến được truyền vào hàm để truyền dữ liệu từ bên ngoài vào bên trong hàm.

Khi gọi hàm, giá trị truyền vào được gọi là đối số (arguments).

✅ Cú pháp:

```
function greet(name) {
  console.log("Xin chào " + name);
}

greet("Quang"); // => Xin chào Quang

// Có thể truyền nhiều tham số
function add(a, b) {
  console.log("Tổng là:", a + b);
}

add(3, 5); // => Tổng là: 8
```

🧩 4. Giá trị trả về (return)
✅ Khái niệm:
Một hàm có thể trả về một giá trị bằng từ khóa return.

Khi return được thực hiện, hàm kết thúc ngay lập tức.

✅ Cú pháp:

```
function nhânHai(x) {
  return x * 2;
}

let kếtQuả = nhânHai(10);
console.log(kếtQuả); // => 20
```

📌 Ghi chú quan trọng:

Mục	Ý nghĩa
`function`	Từ khóa khai báo hàm
`()`	Dấu ngoặc để truyền tham số
`return`	Trả về kết quả của hàm

📘 PHẦN 3: HÀM ẨN DANH & ARROW FUNCTION

🧩 PHẦN 1: HÀM ẨN DANH (Anonymous Function)
🔎 1. Khái niệm lý thuyết

- Hàm ẩn danh là hàm không có tên khi khai báo.
- Thường được gán vào biến, truyền làm tham số, hoặc dùng trong callback.
- Rất phổ biến trong JavaScript hiện đại.

✅ 2. Cú pháp

```
const sayHello = function() {
  console.log("Xin chào!");
};
sayHello(); // Gọi hàm thông qua biến
```

🔍 3. Ứng dụng phổ biến
🔹 a. Gán cho biến:

```
const tinhTong = function(a, b) {
  return a + b;
};

console.log(tinhTong(2, 3)); // 5
```

🔹 b. Truyền làm đối số (callback):

```
setTimeout(function() {
  console.log("Đã chờ 2 giây");
}, 2000);
```

🔹 c. Dùng trong forEach:

```
[1, 2, 3].forEach(function(number) {
  console.log(number * 2);
});
```

📌 Ghi nhớ:

- Hàm ẩn danh không thể tự gọi bằng tên, chỉ có thể gọi thông qua biến/đối tượng chứa nó.
- Có thể dùng để gọn code và tránh trùng tên hàm.

🧩 PHẦN 2: ARROW FUNCTION (HÀM MŨI TÊN)

🔎 1. Khái niệm lý thuyết

- Arrow Function là cú pháp viết hàm ngắn gọn hơn, được giới thiệu từ ES6.
- Không có function, thay bằng dấu =>.

✅ 2. Cú pháp cơ bản
🔹 Viết đầy đủ:

```
const tinhBinhPhuong = (x) => {
  return x * x;
};
```

🔹 Viết rút gọn (khi chỉ có 1 biểu thức và 1 tham số):
`const tinhBinhPhuong = x => x * x;`
🔹 Với nhiều tham số:
`const tinhTong = (a, b) => a + b;`
🔹 Không có tham số:
`const xinChao = () => console.log("Chào bạn!");`

🔍 3. So sánh với hàm thường

Tiêu chí	**Hàm thường (`function`)**	**Arrow Function (`=>`)**
Cú pháp	Dài hơn	Ngắn gọn hơn
Gọi lại chính nó	Dễ (vì có tên)	Khó hơn (phải gán cho biến)
Truyền làm callback	Phổ biến	Cực kỳ phổ biến

📝 BÀI TẬP THỰC HÀNH
✏️ Bài tập 1 – Viết hàm ẩn danh
Tạo một biến hello, gán cho nó một hàm ẩn danh in ra "Xin chào học viên!"

Gọi hàm đó ra 2 lần.

✏️ Bài tập 2 – Viết arrow function
Viết arrow function nhanDoi(x) trả về giá trị x * 2.
Viết arrow function kiemTraChanLe(n) trả về chuỗi "Chẵn" hoặc "Lẻ".

📘 Phần 4: PROMISE & ASYNC/AWAIT TRONG JAVASCRIPT

🧩 PHẦN 1: PROMISE
🔎 1. Khái niệm lý thuyết
✅ Promise là gì?

- Promise là một đối tượng đại diện cho một thao tác bất đồng bộ (async operation).
- Nó giống như lời hứa: hoặc là được thực hiện (fulfilled) hoặc là thất bại (rejected).

✅ Trạng thái của Promise:

Trạng thái	Ý nghĩa
`pending`	Đang chờ xử lý
`fulfilled`	Xử lý thành công (`resolve`)
`rejected`	Xử lý thất bại (`reject`)

✅ 2. Cú pháp tạo Promise

```
const myPromise = new Promise((resolve, reject) => {
  // Xử lý logic ở đây
  if (/* thành công */) {
    resolve(kết_quả);
  } else {
    reject(lỗi);
  }
});
```

✅ 3. Sử dụng .then() và .catch()

```
myPromise
  .then((result) => {
    console.log("Thành công:", result);
  })
  .catch((error) => {
    console.log("Thất bại:", error);
  });
```

📌 Ví dụ minh họa:

```
const sayHello = new Promise((resolve, reject) => {
  resolve("Xin chào!");
});

sayHello.then((message) => {
  console.log(message); // 👉 Output: Xin chào!
});
```

🧠 Ghi nhớ:

- .then() xử lý khi Promise thành công
- .catch() xử lý khi có lỗi
- .finally() chạy sau tất cả (thành công hoặc thất bại)

🧩 PHẦN 2: ASYNC / AWAIT
🔎 1. Khái niệm lý thuyết

```
async/await là cách viết ngắn gọn, dễ hiểu hơn cho Promise.
await chỉ dùng được bên trong hàm có khai báo async.
```

✅ 2. Cú pháp

```
async function tenHam() {
  try {
    const kếtQuả = await lờiHứa;
    console.log(kếtQuả);
  } catch (lỗi) {
    console.error(lỗi);
  }
}
```

📌 Ví dụ minh họa:

```
const getData = () => {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("Dữ liệu đã tải xong!");
    }, 1500);
  });
};

async function main() {
  console.log("Bắt đầu tải...");
  const result = await getData();
  console.log(result);
  console.log("Hoàn tất.");
}

main();
```

🔍 So sánh Promise vs async/await

Tiêu chí	Promise (then/catch)	async/await
Độ dài mã	Dài hơn nếu lồng nhiều `.then()`	Ngắn gọn, dễ đọc hơn
Xử lý lỗi	`.catch()`	`try...catch`
Độ phổ biến	Rộng rãi (chuẩn ES6 trở lên)	Phổ biến hơn sau ES2017

📚 TỔNG KẾT

Nội dung	Ý nghĩa
`Promise`	Xử lý bất đồng bộ, có 3 trạng thái
`then/catch`	Xử lý kết quả thành công hoặc lỗi
`async/await`	Viết gọn gàng hơn, dễ đọc hơn
`try...catch`	Dùng để bắt lỗi khi `await` thất bại

📝 BÀI TẬP THỰC HÀNH

✏️ Bài 1 – Tạo Promise đơn giản

Viết một Promise giả lập việc tải tài liệu:

Sau 2 giây, resolve("Tải thành công!")

Nếu isFail = true, thì reject("Tải thất bại!")

✏️ Bài 2 – Dùng .then() và .catch()

Sử dụng Promise trên:

In ra "✅" nếu thành công, "❌" nếu thất bại

✏️ Bài 3 – Dùng async/await

Viết một hàm async để gọi Promise trên bằng await, dùng try...catch để in kết quả.